

問題1

下記の実行結果となるように、未完成プログラムの空欄部分を埋めてプログラムを完成させてください。

◆文字列操作の各種処理を、それぞれ別々のゴルーチンを起動して行います。

実行結果

```
C:\work\go>go run ensu_0612.go
main関数開始
ゴルーチン開始
"Hello" を大文字にすると "HELLO"
ゴルーチン終了
ゴルーチン開始
"Hello" を小文字にすると "hello"
ゴルーチン終了
ゴルーチン開始
"Hello" の長さは 5
ゴルーチン終了
main関数終了
```

未完成プログラム ファイル名:enshu_0612.go

```
package main
```

```
import (  
    "fmt"  
    "strings"  
    "time"  
)
```

```
/**
```

```
TextOp 構造体の定義
```

```
フィールド:
```

```
    Text 文字列(型 string)
```

```
*/
```

プログラム作成部分

```
/**
```

```
process メソッドの定義
```

```
メソッド名:process
```

```
レシーバ:TextOp 構造体のポインタ
```

```
引数:処理種別 op(型 string)
```

```
処理:
```

```
    指定された 処理種別 op の種類により、各種処理を実行し、実行結果を表示
```

```
    処理種別:
```

```
        len ⇒ TextOp 構造体の文字列の長さを出力
```

```
        toUpper ⇒ TextOp 構造体の文字列を大文字にして出力
```

```
        toLower ⇒ TextOp 構造体の文字列を小文字にして出力
```

```
戻り値:なし
```

```
*/
```

プログラム作成部分

```
func main() {  
  
    // プログラム開始の表示  
    fmt.Println("main 関数開始")  
  
    // TextOp 構造体の変数を宣言  
    _____(1)_____  
  
    // 操作対象の文字列を設定  
    textOp.Text = "Hello"  
    // それぞれの操作をゴルーチンで並列実行  
    // 文字列の長さを出力  
    _____(2)_____  
    // 文字列を大文字にして出力  
    _____(3)_____  
    // 文字列を小文字にして出力  
    _____(4)_____  
  
    // ゴルーチンが完了するまで待機  
    time.Sleep(time.Second)  
  
    fmt.Println("main 関数終了") // プログラム終了の表示  
  
}
```

問題2

問題 1 のプログラムにおいて、チャンネルを利用して、文字列操作の各種処理を表示するように、修正します。

下記の実行結果となるように、未完成プログラムの空欄部分を埋めてプログラムを完成させてください。

実行結果

```
C:\work\go>go run enschu_0612_2.go
main関数開始
ゴルーチン：演算 [toLowerCase] 開始
ゴルーチン：演算 [len] 開始
len 演算結果をチャンネルに送信
ゴルーチン終了
"Hello" の長さは 5
ゴルーチン：演算 [toUpperCase] 開始
toUpperCase 演算結果をチャンネルに送信
ゴルーチン終了
"Hello" を大文字にすると "HELLO"
"Hello" を小文字にすると "hello"
main関数終了
```

```
C:\work\go>go run enschu_0612_2.go
main関数開始
ゴルーチン：演算 [toLowerCase] 開始
toLowerCase 結果をチャンネルに送信
ゴルーチン終了
ゴルーチン：演算 [len] 開始
len 結果をチャンネルに送信
ゴルーチン終了
"Hello" の長さは 5
ゴルーチン：演算 [toUpperCase] 開始
toUpperCase 結果をチャンネルに送信
ゴルーチン終了
"Hello" を大文字にすると "HELLO"
"Hello" を小文字にすると "hello"
main関数終了
```

未完成プログラム ファイル名:enshu_0612_2.go

```
package main
```

```
import (  
    "fmt"  
    "strings"  
)
```

```
/**
```

```
TextOp 構造体の定義
```

```
フィールド:
```

```
    Text 文字列(型 string)
```

```
*/
```

プログラム作成部分

```
/**
```

```
process メソッドの定義
```

```
メソッド名:process
```

```
レシーバ:TextOp 構造体のポインタ
```

```
引数:
```

```
    op 処理種別 (型 string)
```

```
    ch 文字列の送受信チャンネル(型 chan string)
```

```
処理:
```

```
    指定された 処理種別 op の種類により、各種処理を実行し、  
    実行結果を引数として指定されたチャンネルに送信
```

```
    処理種別:
```

```
        len ⇒ TextOp 構造体の文字列の長さを出力
```

```
        toUpper ⇒ TextOp 構造体の文字列を大文字にして出力
```

```
        toLower ⇒ TextOp 構造体の文字列を小文字にして出力
```

```
戻り値:なし
```

```
*/
```

プログラム作成部分

```
func main() {
fmt.Println("main 関数開始") // プログラム開始の表示

_____(1)_____ // TextOp 構造体の変数を宣言
textOp.Text = "Hello" // 操作対象の文字列を設定

// 各操作ごとに個別のチャンネルを作成 バッファ数 10
//文字列の長さ用チャンネル 変数名 chLen
_____(2)_____
//大文字変換用チャンネル 変数名 chUpper
_____(3)_____
//小文字変換用チャンネル 変数名 chLower
_____(4)_____

// 各操作をゴルーチンで実行
/**
    TextOp 構造体の process をゴルーチンとして呼び出し
    引数として "len" と chLen を渡す
*/
_____(5)_____
/**
    TextOp 構造体の process をゴルーチンとして呼び出し
    引数として "toUpper" と chUpper を渡す
*/
_____(6)_____
/**
    TextOp 構造体の process をゴルーチンとして呼び出し
    引数として "toLower" と chLower を渡す
*/
_____(7)_____

// 各チャンネルから結果を個別に受信
fmt.Println(_____(8)_____)
fmt.Println(_____(9)_____)
fmt.Println(_____(10)_____)

fmt.Println("main 関数終了")
}
```